

// Jivops

Guía completa — 30 lecciones

Arquitectura · Ingesta de logs · Almacenamiento · LogQL avanzado ·
Operación y escalado · Integración avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Loki y filosofía 'like Prometheus, but for logs'
- Lección 02 — Arquitectura de Loki — componentes
- Lección 03 — Loki monolithic vs simple scalable vs microservices mode
- Lección 04 — Instalación de Loki (Helm, single binary)
- Lección 05 — LogQL — fundamentos del lenguaje de consulta

Módulo 2: Ingesta de logs

- Lección 06 — Promtail — agente de recolección de logs
- Lección 07 — Labels — diseño de labels eficiente
- Lección 08 — Cardinalidad de labels — el error más común
- Lección 09 — Pipelines de Promtail — parsing y transformación
- Lección 10 — Alternativas a Promtail: Fluent Bit, Vector, Grafana Agent

Módulo 3: Almacenamiento

- Lección 11 — Chunks e índices — cómo almacena Loki los datos
- Lección 12 — Backend de almacenamiento (S3, GCS, filesystem)
- Lección 13 — TSDB index — el índice moderno de Loki
- Lección 14 — Retención de logs y compactación
- Lección 15 — Costos de almacenamiento vs Elasticsearch

Módulo 4: Consultas LogQL avanzado

- Lección 16 — Filtros de línea (line filters)
- Lección 17 — Parsers en LogQL (json, logfmt, regexp)
- Lección 18 — Métricas desde logs con LogQL
- Lección 19 — Agregaciones y funciones de rango
- Lección 20 — Optimización de queries lentas en Loki

Módulo 5: Operación y escalado

- Lección 21 — Escalado horizontal de Loki (simple scalable mode)
- Lección 22 — Multi-tenancy en Loki
- Lección 23 — Rate limiting y protección contra sobrecarga
- Lección 24 — Monitoreo del propio Loki (metamonitoring)
- Lección 25 — Troubleshooting de ingesta fallida

Módulo 6: Integración avanzada

Lección 26 — Integración con Grafana — exploración y dashboards

Lección 27 — Alerting basado en logs (Loki Ruler)

Lección 28 — Integración con Kubernetes (labels automáticos)

Lección 29 — Migración desde Elasticsearch/ELK a Loki

Lección 30 — Proyecto final: stack de logging completo con Loki listo para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Loki y filosofía 'like Prometheus, but for logs'

Loki no indexa el contenido completo del texto, solo un conjunto reducido de labels, haciendo el índice mucho más liviano y económico que Elasticsearch.

Lección 02 — Arquitectura de Loki — componentes

Separar distributor, ingester y querier permite escalar cada uno según su carga específica: mucha escritura no siempre implica muchas consultas.

Lección 03 — Loki monolithic vs simple scalable vs microservices mode

El modo 'simple scalable' divide lectura y escritura, dando capacidad de escalar cada una sin la complejidad completa del modo microservices.

Lección 04 — Instalación de Loki (Helm, single binary)

El Helm chart oficial ya resuelve gran parte de la configuración inicial recomendada, reduciendo el riesgo de errores manuales.

```
helm install loki grafana/loki
```

Lección 05 — LogQL — fundamentos del lenguaje de consulta

Toda consulta empieza con un selector de labels, aprovechando el índice liviano de Loki antes de aplicar filtros de texto más costosos.

```
{app="miapp"} |= "error"
```

Módulo 2: Ingesta de logs

Lección 06 — Promtail — agente de recolección de logs

Promtail detecta automáticamente nuevos Pods vía service discovery de Kubernetes, sin requerir reconfiguración manual por cada despliegue nuevo.

Lección 07 — Labels — diseño de labels eficiente

Los labels deben tener un número limitado y estable de combinaciones posibles, como namespace o nivel de log, no valores únicos por línea.

Lección 08 — Cardinalidad de labels — el error más común

Un label como 'user_id' con miles de valores generaría miles de streams separados, sobrecargando gravemente el sistema de índices.

Lección 09 — Pipelines de Promtail — parsing y transformación

Extraer el nivel de log con un stage regex y convertirlo en label permite filtrar rápidamente por índice, en vez de buscar texto en cada línea.

Lección 10 — Alternativas a Promtail: Fluent Bit, Vector, Grafana Agent

Un equipo que ya usa Fluent Bit para otro sistema puede preferir seguir usándolo, evitando duplicar agentes de recolección distintos.

Módulo 3: Almacenamiento

Lección 11 — Chunks e índices — cómo almacena Loki los datos

Agrupar líneas en un chunk comprimido reduce drásticamente las operaciones de escritura individuales, comprimiendo más eficientemente en bloque.

Lección 12 — Backend de almacenamiento (S3, GCS, filesystem)

Depender de filesystem local en un solo nodo es un riesgo grave: si ese nodo falla, todos los logs ahí almacenados se perderían sin redundancia.

```
bucketnames: mis-logs-loki
```

Lección 13 — TSDB index — el índice moderno de Loki

Las mejoras de eficiencia de TSDB tienen un impacto proporcionalmente mayor cuanto más grande es el volumen de datos gestionado.

Lección 14 — Retención de logs y compactación

El compactador elimina automáticamente chunks e índices que superan el período de retención, evitando que el almacenamiento crezca sin control.

```
retention_period: 720h
```

Lección 15 — Costos de almacenamiento vs Elasticsearch

El ahorro relativo de un índice mucho más pequeño se vuelve proporcionalmente mayor cuanto más crece el volumen total de logs gestionado.

Módulo 4: Consultas LogQL avanzado

Lección 16 — Filtros de línea (line filters)

Aplicar un filtro de texto barato (`|=`) antes de un parser costoso reduce rápidamente el conjunto de líneas relevantes, mejorando el rendimiento general.

Lección 17 — Parsers en LogQL (json, logfmt, regexp)

El parser debe coincidir con el formato estructurado real del log: `logfmt` para `key=value`, `json` para JSON, no son intercambiables.

```
{app="miapp"} | json
```

Lección 18 — Métricas desde logs con LogQL

Una `metric query` convierte líneas de log en una serie temporal numérica graficable y alertable, algo difícil de lograr directamente con texto.

```
rate({app="miapp"} |= "error" [5m])
```

Lección 19 — Agregaciones y funciones de rango

Agrupar `'by (app)'` identifica exactamente qué servicio genera más errores, en vez de un número total que mezcla todos los servicios.

Lección 20 — Optimización de queries lentas en Loki

Una consulta con `label` amplio y `range` de tiempo largo combina el peor escenario posible: muchos streams y mucho período de datos a procesar.

Módulo 5: Operación y escalado

Lección 21 — Escalado horizontal de Loki (simple scalable mode)

Escalar el path de escritura o lectura según la demanda real evita desperdiciar recursos en componentes que no reciben tanta carga proporcionalmente.

```
kubectl scale deployment loki-write --replicas=3
```

Lección 22 — Multi-tenancy en Loki

El header X-Scope-OrgID mantiene los datos de cada tenant lógicamente aislados, compartiendo infraestructura de forma eficiente entre clientes.

Lección 23 — Rate limiting y protección contra sobrecarga

Un límite de ingestion_rate por tenant protege a los demás tenants de que uno con volumen anormal degrade el servicio compartido.

```
ingestion_rate_mb: 10
```

Lección 24 — Monitoreo del propio Loki (metamonitoring)

Sin monitorear las métricas internas de Loki, un problema de ingesta podría pasar desapercibido hasta que se necesite un log que nunca se guardó.

Lección 25 — Troubleshooting de ingesta fallida

Un error 429 al enviar logs desde Promtail indica que se alcanzó un límite de rate limiting configurado en Loki.

Módulo 6: Integración avanzada

Lección 26 — Integración con Grafana — exploración y dashboards

Correlacionar métricas de Prometheus y logs de Loki en la misma interfaz acelera la investigación, sin saltar entre herramientas distintas.

Lección 27 — Alerting basado en logs (Loki Ruler)

El Ruler puede alertar sobre patrones específicos de la aplicación (como errores 500), algo que no siempre se refleja en métricas tradicionales de infraestructura.

Lección 28 — Integración con Kubernetes (labels automáticos)

Promtail añade automáticamente labels como namespace y pod, permitiendo filtrar de inmediato un incidente sin configuración manual previa.

Lección 29 — Migración desde Elasticsearch/ELK a Loki

Una consulta de Elasticsearch basada en texto libre necesita repensarse con la filosofía de Loki: reducir primero por labels antes de buscar texto.

Lección 30 — Proyecto final: stack de logging completo con Loki listo para producción

Se integra todo lo anterior: labels bien diseñados, object storage, TSDB index, retención configurada, escalado horizontal, alerting con Ruler y metamonitoring — el nivel esperado de un stack de logging con Loki listo para producción real.